Lab 11-1 Convolution Neural Network & Data Pipelines

NTHU DataLab, 2026

Outline

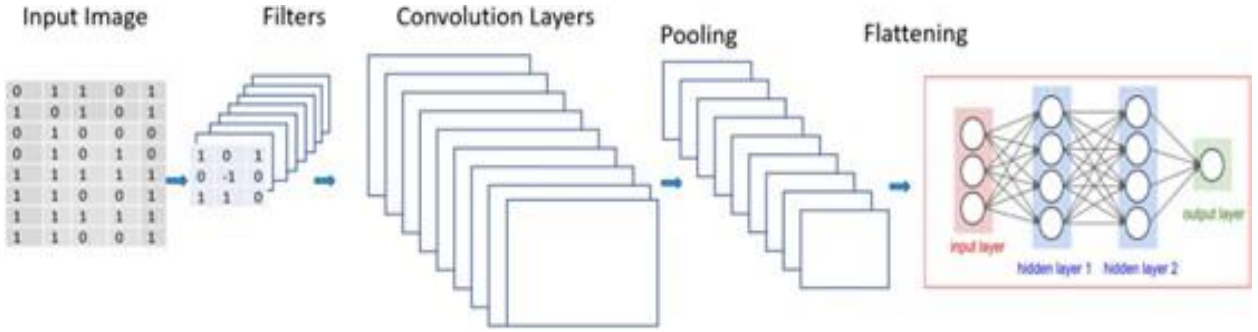
- Convolution neural network
- Input pipeline
- Optimization for Input pipeline (PyTorch)

Outline

- Convolution neural network
- Input pipeline
- Optimization for Input pipeline (PyTorch)

Convolution Neural Network

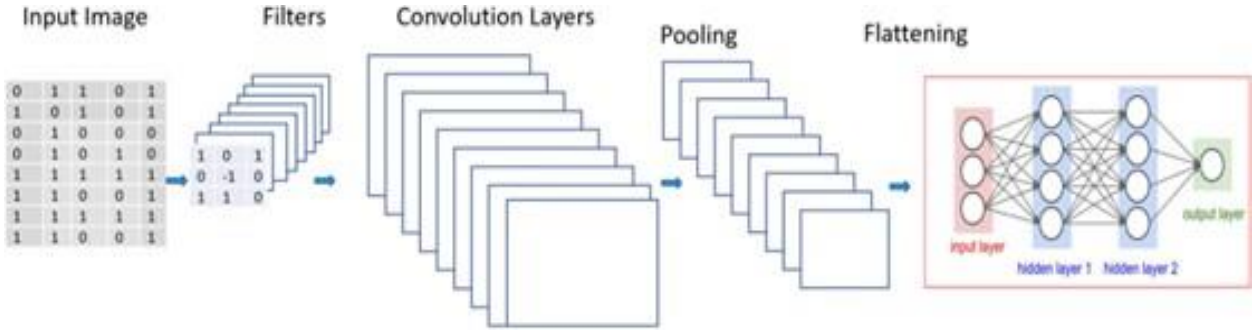
- Build a CNN model architecture in __int__(self)
  - A stack of Conv2D and MaxPooling2D layers




```
class ConvBase2(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=0)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=0)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=0)
```

Convolution Neural Network

- Build a CNN model architecture in __int__(self)
  - A stack of Conv2D and MaxPooling2D layers

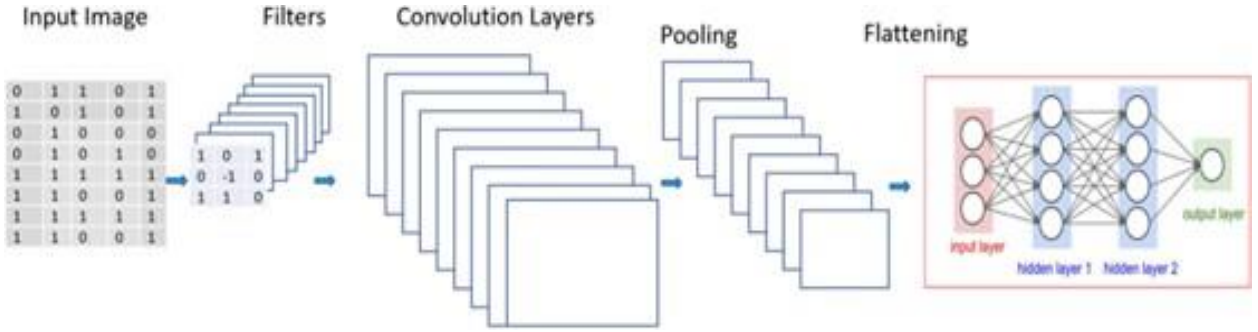



```
class ConvBase2(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=0)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=0)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=0)
```

of Input channel

Convolution Neural Network

- Build a CNN model architecture in __int__(self)
  - A stack of Conv2D and MaxPooling2D layers

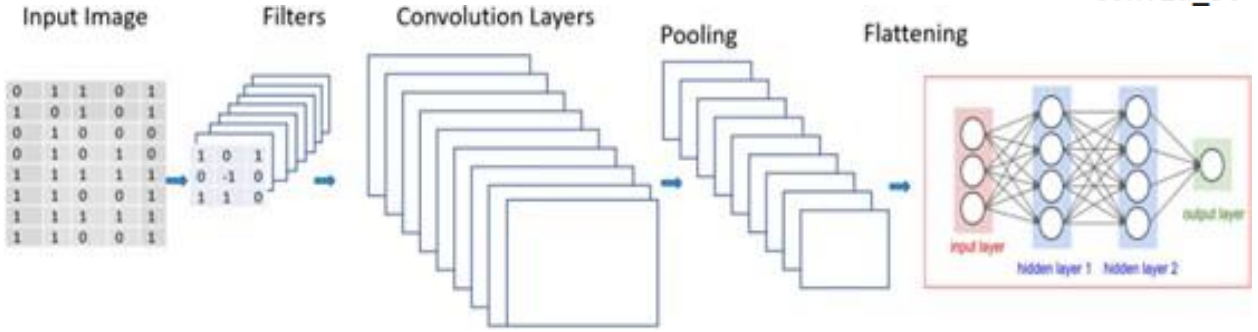



```
class ConvBase2(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=0)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=0)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=0)
```

of output channel = # of filter

Convolution Neural Network

- Build a CNN model architecture in __int__(self)
  - A stack of Conv2D and MaxPooling2D layers

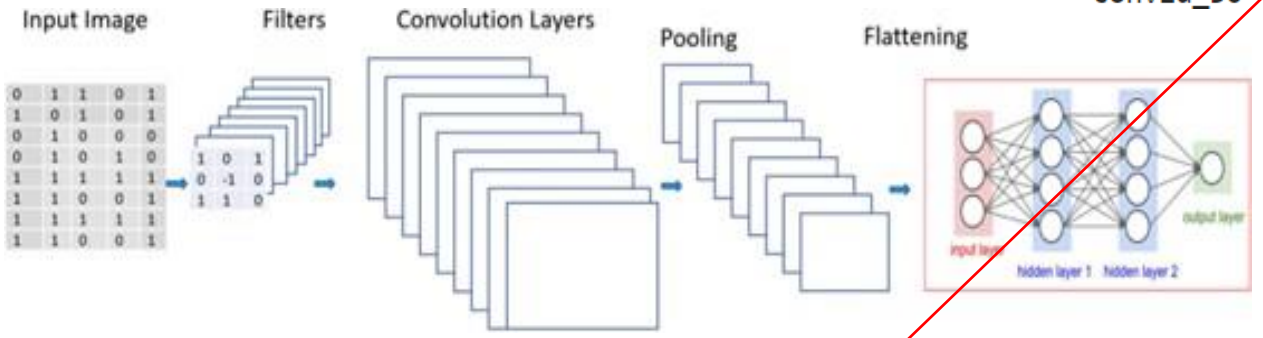


Layer (type)	Output Shape
conv2d_28 (Conv2D)	(None, 26, 26, 32)
max_pooling2d_12 (MaxPooling2D)	(None, 13, 13, 32)
conv2d_29 (Conv2D)	(None, 6, 6, 64)
max_pooling2d_13 (MaxPooling2D)	(None, 3, 3, 64)
conv2d_30 (Conv2D)	(None, 3, 3, 64)

```
class ConvBase2(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=0)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=0)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=0)
```

Convolution Neural Network

- Build a CNN model architecture in __int__(self)
  - A stack of Conv2D and MaxPooling2D layers



Layer (type)	Output Shape
conv2d_28 (Conv2D)	(None, 26, 26, 32)
max_pooling2d_12 (MaxPooling2D)	(None, 13, 13, 32)
conv2d_29 (Conv2D)	(None, 6, 6, 64)
max_pooling2d_13 (MaxPooling2D)	(None, 3, 3, 64)
conv2d_30 (Conv2D)	(None, 3, 3, 64)

```
class ConvBase2(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=0)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=0)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=0)
```

Outline

- Convolution neural network
- Input pipeline
- Optimization for Input pipeline (PyTorch)

Input Pipeline

A typical training data pipeline can be framed as an ETL process:

- **Extract**: Read data from memory (NumPy) or persistent storage -- either local (HDD/SSD) or remote.
- **Transform**: parse and preprocess the data -- shuffling, batching, domain-specific transformations such as image decoding/augmentation.
- **Load**: move the transformed data onto the accelerator device(s) that execute the mode

Input Pipeline - PyTorch

In PyTorch, this job is split across two plain-Python objects:

- **torch.utils.data.Dataset**: exposing __len__ and __getitem__(idx) -- this is the "Extract + per-sample Transform" step. Transformation is done inside __getitem__, often by calling a torchvision.transforms pipeline.
- **torch.utils.data.DataLoader**: wraps a Dataset and handles shuffling, batching and prefetching -- this is the "Load" step. All of these are configured as constructor arguments.

Input Pipeline

- Transformations
  - **Map**: apply the function to each the elements of this dataset. E.g. crop, rotation, flip ….
  - **Shuffle**: maintains a fixed-size buffer and chooses the next element uniformly at random from that buffer
  - **Batch**: combines consecutive elements of this dataset into batches
  - **Prefetch**: allows later elements to be prepared while the current element is being processed

```python
class CifarPickleDataset(Dataset):
    def __init__(self, filenames, train):
        images, labels = [], []
        for filename in filenames:
            image, label = read_file(filename)
            images.append(image)
            labels.extend(label)
        self.images = np.concatenate(images).reshape(-1, IMAGE_DEPTH, IMAGE_HEIGHT, IMAGE_WIDTH)
        self.labels = np.array(labels, dtype=np.int64)

        # equivalent of map_fun / map_fun_test
        aug = [transforms.RandomCrop(IMAGE_SIZE_CROPPED)] if train else \
              [transforms.CenterCrop(IMAGE_SIZE_CROPPED)]
        if train:
            aug += [
                transforms.RandomHorizontalFlip(),
                transforms.ColorJitter(brightness=63/255, contrast=(0.2, 1.8)),
            ]
        self.transform = transforms.Compose(aug + [
            # per-image standardization: TF's per_image_standardization()
            # subtracts the per-image mean and divides by (approx.) the per-image std
            transforms.Lambda(lambda t: (t - t.mean()) / t.std().clamp_min(1.0 / math.sqrt(t.numel()))),
        ])

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        image = torch.from_numpy(self.images[idx]).float() / 255.0  # already CHW
        image = self.transform(image)
        return image, self.labels[idx]
```

```python
class CifarPickleDataset(Dataset):
    def __init__(self, filenames, train):
        images, labels = [], []
        for filename in filenames:
            image, label = read_file(filename)
            images.append(image)
            labels.extend(label)
        self.images = np.concatenate(images).reshape(-1, IMAGE_DEPTH, IMAGE_HEIGHT, IMAGE_WIDTH)
        self.labels = np.array(labels, dtype=np.int64)

        # equivalent of map_fun / map_fun_test
        aug = [transforms.RandomCrop(IMAGE_SIZE_CROPPED)] if train else \
              [transforms.CenterCrop(IMAGE_SIZE_CROPPED)]
        if train:
            aug += [
                transforms.RandomHorizontalFlip(),
                transforms.ColorJitter(brightness=63/255, contrast=(0.2, 1.8)),
            ]
        self.transform = transforms.Compose(aug + [
            # per-image standardization: TF's per_image_standardization()
            # subtracts the per-image mean and divides by (approx.) the per-image std
            transforms.Lambda(lambda t: (t - t.mean()) / t.std().clamp_min(1.0 / math.sqrt(t.numel()))),
        ])

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        image = torch.from_numpy(self.images[idx]).float() / 255.0  # already CHW
        image = self.transform(image)
        return image, self.labels[idx]
```

Outline

- Convolution neural network
- Input pipeline
- Optimization for Input pipeline (PyTorch)

Parallel loading

- Set **num_workers** > 0 (default : 0) : multiple worker processes read/decode samples concurrently.
- If num_workers > 0 , prefetch is default
- N workers means up to N copies of whatever the dataset holds. How workers get the copy is OS dependent and may sometimes causes error without caution.

Prefetch

No prefetching (num_workers=0):

CPU: [load 1] [load 2] [load 3]

GPU: [train 1] [train 2] [train 3]

time



With prefetching:

CPU: [load 1][load 2][load 3][load 4]

GPU: [train 1][train 2][train 3][train 4]

time



Prefetching

- **prefetch_factor** : The number of batches each worker loads ahead of time.
- **pin_memory** : The GPU copies data using DMA (direct memory access), which reads RAM without the CPU’s involvement. DMA needs the data to stay at a fixed physical address, so it can only read pinned (page-locked) memory
- Note that different batches don’t necessary take same time to fetch

Cache (in Dataset)

- We go over the same data every epoch. Without caching, each epoch repeats the same work for every image: read the file from disk, decode it, convert it to a tensor.
- Cache the decoded image, then transform
- Note that workers restart every epoch, The new workers start from a copy of the dataset in the main process, whose cache is empty.
- Even with **persistent_workers=True**, the caches are split across worker because of shuffle